LIVE TRACK

Real-time bus tracking, parent-first

System Guide & Testing Handbook

What the platform does, how each user experiences it, and how to test it end-to-end.

Wednesday, 9 September 2026

For the pilot with the Kigali school + anyone joining the team.

What is Live Track?

Live Track is a real-time bus tracking platform for schools. Every driver's phone becomes a live GPS device -- no separate hardware, no SIM per vehicle, no installer visit. The driver opens the app, taps Start, and their bus streams live to the school's admin dashboard and to every parent linked to a child on that bus.

The platform was built first for a Kigali school in closed pilot, but every feature -- driver app, admin dashboard, parent map -- adapts to any operator that needs live visibility over people they trust with vehicles. Delivery, ride-share, field teams: same system, same phones.

The three questions Live Track answers

- PARENT: Where is my child's bus right now, and when will it arrive at our stop?
- ADMIN: Where is every bus on my fleet right now, and what happened yesterday?
- DRIVER: How do I share my position without extra hardware or a data-hungry setup?

What lives where

Frontend: TanStack Start (React 19) + Vite, deployed on Vercel via a Node serverless function (api/ssr.js).

Map: Leaflet + react-leaflet with CARTO Voyager tiles, theme-aware (works in light and dark).

Backend: Supabase (Postgres + Auth + Row Level Security + Realtime). All parent-vs-parent isolation is enforced at the database layer, so bugs in the client cannot leak another family's data.

Names on the map: OpenStreetMap Nominatim (reverse geocoding), rate-limited to 1 request per second and cached to respect fair use.

Server-only writes (parent-account creation, service-role admin ops): api/parent-redeem.js on Vercel; identical Vite dev middleware locally.

The four kinds of users

Everything in the app is scoped to WHO you are. Roles live in the user_roles table and cannot be changed from the client.

Role	What they do	Where they land
Super admin (admin)	Run everything. Can see every school and every bus.	/dashboard, /admin/*, /history
School admin	Runs ONE school. Manages routes, stops, buses, drivers, students, invites.	/admin/routes, /admin/buses, /admin/students, /admin/invites
Driver (user)	Owns a bus. Their phone is the tracker.	/track
Parent	Watches ONE bus (their child's). Cannot see anything else.	/join (first time), /me (always)

The trust model in one paragraph

A parent can read one row from every relevant table -- their own child, their own child's bus, that bus's route and stops, and the device_status row of the driver of that bus. They cannot read any other row. They cannot write anything. The rules live as Postgres RLS policies, verified by 18 assertions in scripts/test-parent-isolation.sql (all pass).

The data model in plain English

The database has 12 tables that matter. The relationships:

- SCHOOL has many ROUTES. A ROUTE has many STOPS in a sequence (Kimironko -> Nyarugenge -> School).
- SCHOOL has many BUSES. Each BUS is assigned a ROUTE and a DRIVER (a user account with role='user').
- SCHOOL has many STUDENTS. Each STUDENT is assigned a BUS and a home STOP.
- An INVITE code created by an admin lists one or more STUDENT ids. Redeeming it creates a PARENT account.
- PARENT_STUDENTS links a PARENT (auth user) to one or more STUDENTS. This is the ONLY thing that grants read access.

Live tracking tables

- device_status: one row per driver, always UPSERTed with their latest position, speed, battery, last_seen.
- location_history: one row per GPS point the driver's phone uploads (~every 8 seconds).
- trip_sessions: one row per Start/Stop button press. Open when ended_at is null.
- geofence_events: written server-side by a DB trigger when the bus enters or leaves a stop's radius. This is what drives 'Approaching' and 'Arrived' banners on the parent screen.

How presence works

Fresh vs offline is derived from device_status.last_seen:

- Less than 30 seconds ago -> the bus is live. Show its stored status (moving / stopped / idle).
- More than 90 seconds ago -> the bus is offline. Show 'Bus is offline -- last seen at HH:MM near <place>'.
- Between 30 and 90 seconds -> trust the stored status only if it's clearly moving or stopped; otherwise call it offline. This is the guard against showing a frozen 'live' marker.

Feature tour: the driver (/track)

The driver opens the app, signs in with email + password, and lands on /track. It has one primary button and a lot of status.

What the driver sees

- Big Start / Stop button.
- Status pill: Moving / Parked / Idle / Offline with a coloured dot.
- Speed, battery, GPS accuracy, whether they're online, whether GPS is active.
- A permission prompt on first tap of Start; if they deny it, the app tells them how to re-grant.

What happens under the hood

- On Start: opens Wake Lock so the screen stays on for the shift; starts watchPosition; opens a trip_sessions row (ended_at = null).
- Every ~8 seconds while moving: upserts device_status (latest lat/lng, speed, battery) and appends one row to location_history.
- On Stop: releases Wake Lock, sets device_status.status='offline', clears the tracking flag, and closes the trip_session (ended_at = now()).
- On Sign out: same offline-flip runs BEFORE the auth session is torn down. Without this the driver would look 'online' to admins for another minute.
- The Kigali -> Kampala 'ghost drive' bug is fixed. The trip report never fabricates a jump: any pair of points separated by more than 3 minutes OR whose implied speed exceeds 150 km/h is flagged 'Signal lost' and contributes zero km and zero minutes.

WATCH

Real-phone caveat

Automated tests can prove the code is correct. They CANNOT prove that a real Android will keep watchPosition alive with the screen locked / browser backgrounded / OEM battery-optimiser active. That must be tested on a real phone on a real route.

Feature tour: the admin (/dashboard and /admin/*)

/dashboard -- the live fleet map

- Every driver as a coloured marker (moving / stopped / idle / offline).
- Filter chips: All / Moving / Parked / Idle / Offline. The MAP still shows every driver even when the list is filtered -- filters only narrow the side list.
- KPIs in the header: number Active vs number Offline.
- Click a driver to fly the map to them and see plate, driver name, address, battery, phone on/off, and 'updated N seconds ago'.
- Admins and school_admins never appear here -- the list is joined to user_roles.role='user' server-side.

/history -- day playback + PDF report

- Pick a driver + a date. See the polyline of their route, every stop, and a summary (distance, driving time, stopped time, top speed).
- Export PDF: one-click daily report. Header carries district name, then Start/End with named places, a Signal-Lost section listing any gaps, a Stops table with human street names, and a route narrative -- NO raw lat/lon anywhere in the UI.

/admin/routes -- build the routes and stops

- Create a route. Add stops in order: name, click a Leaflet map to set lat/lng, adjust the radius (default 120 m). Stops can be type 'pickup' or 'school'.
- Reorder stops with up/down arrows; delete stops.

/admin/buses -- provision each bus

- Set label + plate + capacity, assign a route, assign a driver (the picker only lists accounts with role='user').

/admin/students -- roster

- Add students. Assign each to a bus and to a home stop; the home-stop picker is limited to the stops on that bus's route.

/admin/invites -- generate parent codes

- Pick one or more students -> generate an 8-character code from an unambiguous alphabet (no 0, O, 1, I, L).
- Optional parent phone (7-15 digits) and optional expiry (must be in the future). Live error message + disabled Generate button until the form is valid.
- List existing codes with 'unused / used / expired' badges. Copy a code to clipboard, or revoke an unused one.

Feature tour: the parent (/join and /me)

/join -- first time

- One input: the 8-char code from the school. Client-side check rejects wrong-alphabet characters (catches 'O typed instead of 0') before hitting the server.
- Next screen: set a 4-digit PIN, confirm it once. Server creates the parent's auth user (email = normalised phone or code + '@parent.pulse.local', email_confirm: true so no email is ever sent), links parent_students, burns the invite so it can't be reused, and signs the parent in.
- Persisted session: on subsequent visits, /join auto-forwards to /me without asking again.

/me -- the live parent screen

- Big status banner, colour-coded: 'Bus is moving' / 'Bus is stopped' / 'Bus is offline' / 'No trip yet today'.
- Live map: bus marker, route line, every named stop labelled, home stop and school stop emphasised (larger, coloured). Passed stops fade to grey.
- Trip progress row: chips for each stop -- checked for done, pulsing for current, empty for pending.
- ETA line: 'Arriving in ~7 min (approx)' or 'Arriving now' when close, or 'Passed your stop' after the bus has left home stop this trip. Never negative. Speed is a rolling average of the last 5 minutes; fallback 20 km/h if not enough points.
- Trust row: bus plate, driver name, 'updated Ns ago' (5-second ticker).
- Two proactive banners driven by geofence_events: 'Approaching' fires on arrive at the stop BEFORE home; 'Arrived at your stop' fires on arrive at home.
- OS-level notifications (Notify me toggle) if the parent grants permission -- delivers even when the tab is backgrounded.

Multi-child on the same bus

If a parent has siblings on one bus, /me groups them into one chip labelled 'Alice + Bob' and shows 'Alice and Bob's bus arrived at ...' in banners -- not two identical maps side by side.

NOTE

What is NOT on /me yet

True web-push notifications when the tab is fully CLOSED are a Phase 5 item -- toggle + storage + linkage guard are all in place; VAPID keys + service worker + edge-function sender are the missing pieces. WhatsApp notifications are also deferred (needs registering a WhatsApp Business number for the Rwanda authentication/utility rate).

How to test, end-to-end

Fresh-pilot DB is already loaded: 4 users (2 admins + 2 drivers), 1 school, 1 route, 4 stops, 2 buses, 4 students. No parent accounts and no tracking data. Everything below assumes you're pointed at that state.

Scenario A -- admin signs in and inspects the school

- Go to /auth. Sign in as one of the admin accounts (e.g. shemaabdul80@gmail.com or elogearts3@gmail.com).
- Land on /dashboard. It's empty of live drivers (no one is tracking yet) -- that is CORRECT for a fresh state.
- Click 'School setup' -> /admin/routes. You should see 'Route A - Morning' with 4 stops (Kimironko, Nyarugenge, School and one added earlier).
- Click /admin/buses. You see Bus Isabuku (driver Guy) and Bus Umuyaga (driver Shema).
- Click /admin/students. You see 4 students, each assigned to a bus and a home stop.

Scenario B -- admin generates a parent invite

- /admin/invites -> pick one student -> Generate. A code appears (e.g. 'K4M2X7P9').
- Try adding phone 'abc' -- form should block with 'Phone should be 7-15 digits'.
- Try setting Expires to yesterday -- form should block with 'Expiry must be in the future'.
- Copy the code to clipboard using the copy button next to it.

Scenario C -- parent joins in a private window

- Open a new private/incognito window (so it doesn't share your admin session).
- Go to /join -> paste the code -> Continue -> set PIN 1234 -> confirm 1234 -> Redeem & sign in.
- You should land on /me. Since no bus is currently tracking, the banner reads 'No trip yet today'.

Scenario D -- driver simulates a shift

- In yet another window, sign in as a driver (e.g. elogechris4@gmail.com if you have the password).
- Land on /track. Tap Start tracking; grant location permission when prompted.
- Return to the parent's /me window -- within a few seconds the banner flips to 'Bus is moving' or 'Bus is stopped', and the bus marker appears on the map with 'updated Ns ago' ticking.
- Have the driver walk (or, in dev, use browser devtools to inject fake coordinates) toward the home stop's coordinates -- the parent's ETA counts down. When the driver enters the home stop's radius, /me flashes an 'Arrived at your stop' banner and (if Notify me is on) an OS notification appears.
- Driver taps Stop. Within seconds the parent's banner flips to 'Bus is offline'.

Scenario E -- isolation self-check

- In one private window sign in as Parent A. In another private window sign in as Parent B (needs a second invite for a different bus).
- Confirm each parent only sees their own child's bus, driver, and route -- Parent A must NOT see Parent B's bus at any point.
- This same guarantee is enforced by 18 SQL assertions in scripts/test-parent-isolation.sql (run: psql "$SUPABASE_DB_URL" -f scripts/test-parent-isolation.sql).

Operations

Prerequisites you MUST have set

WATCH

Supabase Dashboard: Confirm email OFF

Authentication -> Providers -> Email -> untick 'Confirm email'. Parent emails are @parent.pulse.local (not deliverable). If Confirm email is on, the parent gets stuck waiting for an email that will never arrive.

WATCH

Vercel env: SUPABASE_SERVICE_ROLE_KEY

Vercel -> Project Settings -> Environment Variables -> add SUPABASE_SERVICE_ROLE_KEY with the sb_secret_... value from Supabase (Settings -> API -> Secret keys). Without it, /api/parent-redeem returns 'The service isn't ready yet'.

Two reset scripts

scripts/reset-tracking-only.sql -- clears only GPS/trip/event/prefs data. Keeps school setup and all auth users. Safe before any pilot.

scripts/reset-fresh-pilot.sql -- also deletes every parent user + invite + parent_students. Keeps the 4 non-parent users and the school. This is the current state.

scripts/reset-full-wipe.sql -- DESTRUCTIVE (deletes every auth user including you). Wrapped in a commented DO block so accidental paste-and-run is inert. Includes step-by-step recovery instructions at the bottom.

How the app talks about errors

Users never see raw Postgres or Supabase error strings. Every toast in the app runs through src/lib/friendly-errors.ts, which maps common technical messages to short calm sentences:

What the DB said	What the user sees
new row violates row-level security policy for table ...	You don't have access to that. Please refresh and try again.
Invalid login credentials	That phone or PIN doesn't match. Check both and try again.
Email not confirmed	This account isn't ready yet. Please contact your school.
JWT expired / unauthorized	Your session has expired. Please sign in again.
duplicate key value violates unique constraint	That entry already exists.
Failed to fetch / network / net::	Can't reach the server. Check your connection and try again.
Server misconfigured: SUPABASE_URL or SUPABASE_SERVICE_ROLE_KEY missing	The service isn't ready yet. Please try again in a few minutes.

Raw messages still go to console.error so developers can debug them.

What's proven, what still needs a real phone

Automated -- proven and reproducible

Suite	Passed
scripts/test-parent-trip.ts (projection, ETA, banner mapping)	**27 / 27**
scripts/test-parent-presence.ts (fresh vs offline)	**12 / 12**
scripts/test-fixes.ts (Kigali-gap distance, presence)	**12 / 12**
Parent isolation SQL (Parent A vs Parent B)	**7 / 7**
Cross-school RLS SQL (school A admin vs school B)	**3 / 3**
Used / expired code SQL	**2 / 2**
Geofence + isolation e2e SQL (Kimironko -> Nyar -> School)	**6 / 6**
Total automated	**69 / 69**

Must be tested on a real phone on a real route

- Whether the browser keeps watchPosition alive with the screen locked. This is device- and OEM-specific.
- Whether the OEM battery optimiser kills the tab after 10-15 minutes.
- Whether backgrounding the browser (opening WhatsApp then coming back) resumes upload cleanly.
- Whether the OS notification delivers when the tab is backgrounded vs closed (true page-closed push is Phase 5 -- needs VAPID + service worker + edge-function sender).
- Whether Realtime latency in Kigali under 3G/4G is acceptable for parents.

TIP

Recommended real-world dry run

Sign in a driver on a real phone. Take the phone in your pocket on a full route with the screen off. Meanwhile a parent watches /me on their phone. At the end, look at /history and check the polyline matches the route you actually drove.

Quick reference

URLs at a glance

URL	Who	What
/	Anyone	Landing page. Nav has 'Parent', 'Sign in', 'Access workspace'.
/auth	Admin, Driver	Email + password sign in / sign up. Not for parents.
/track	Driver	Start / Stop tracking, status pill, battery.
/dashboard	Super admin	Live map of the whole fleet.
/history	Super admin	Any driver + any date -> route + PDF report.
/admin/routes	Admin, School admin	Routes and named stops (with Leaflet stop picker).
/admin/buses	Admin, School admin	Buses; assign route + driver.
/admin/students	Admin, School admin	Students; assign bus + home stop.
/admin/invites	Admin, School admin	Generate + revoke parent codes.
/join	Parent	Enter code -> set PIN -> sign in. Returning-parent tab too.
/me	Parent	Live bus map, ETA, arrival banners, notify toggle.
/api/parent-redeem	Server	Serverless endpoint that redeems a code. Never called directly.

If a parent is stuck

- 'That code doesn't match any invite' -> admin regenerates one on /admin/invites, hand parent the new code.
- 'This code has already been used' -> the parent already redeemed once; ask them to use their phone + PIN on /join's returning-parent tab, or issue a fresh code.
- 'Your session has expired' -> tell them to close and reopen the tab; if it persists, use the returning-parent tab.
- 'You don't have access to that' -> refresh the page; if it persists, ask the admin to check parent_students has the row.
- Bus stays 'Bus is offline' -> confirm on /dashboard that the driver is actually tracking; check the driver's phone battery and permission.

Ship notes

- Merge test/full-system -> main to deploy. Vercel auto-deploys the moment main changes.
- Set SUPABASE_SERVICE_ROLE_KEY on Vercel BEFORE the deploy lands.
- Confirm the Supabase 'Confirm email' toggle is OFF.
- First live test after deploy: create an invite as admin, redeem as parent in a private window, watch /me update as the driver moves.